

AGENTIC AI — LECTURE 3

RAG — Giving Agents Real Knowledge

Retrieval Augmented Generation, Explained Simply

Theory + Practical • VS Code • Groq (100% Free)

AGENDA

Today's Roadmap

1. Quick Recap: What Is RAG?
2. The Problem RAG Solves
3. Embeddings — Turning Text Into Numbers
4. Vector Stores — Searching by Meaning
5. The Full RAG Flow
6. Practical: A Company FAQ Bot
7. Asking Questions With RAG
8. Recap + Q&A

WHERE WE LEFT OFF

Quick Recap — What Is RAG?

Back in Lecture 1, we touched on this idea: RAG means giving the LLM extra, up-to-date information to read before it answers.



RAG = Retrieval Augmented Generation

"Retrieval" = fetching relevant information

"Augmented" = adding it to what the LLM already knows

"Generation" = the LLM writes the final answer using that information

Today, we stop talking about it and actually BUILD one, step by step.

PART 1 — WHY RAG?

The Problem RAG Solves

An LLM only knows what it was trained on. It has no idea about your private documents, your company's policies, or anything that happened after its training.

Without RAG

"What is our warranty policy?"

"I don't have access to your company's specific policies."

With RAG

"What is our warranty policy?"

"Based on your documents: panels have a 10-year warranty, batteries have 5 years."

Embeddings — Turning Text Into Numbers



An embedding is a list of numbers that captures the MEANING of a piece of text.

```
"warranty period"        →  [0.12, -0.45, 0.88, ... ]

"guarantee length"       →  [0.14, -0.42, 0.85, ... ]
```

Notice: these two phrases mean almost the same thing — so their number lists end up very close to each other too!

This is exactly what lets a computer find text that MEANS the same thing, even if the exact words are different.

Vector Stores — Searching by Meaning



A vector store is simply a database built to store embeddings, and quickly find the ones most SIMILAR to a new question.

How it works in 3 simple steps:

1. Store	Every chunk of your document is turned into an embedding and saved.
2. Ask	Your question is ALSO turned into an embedding, the same way.
3. Match	The vector store finds the stored chunks whose embeddings are closest to your question's embedding.

PART 1 — PUTTING IT TOGETHER

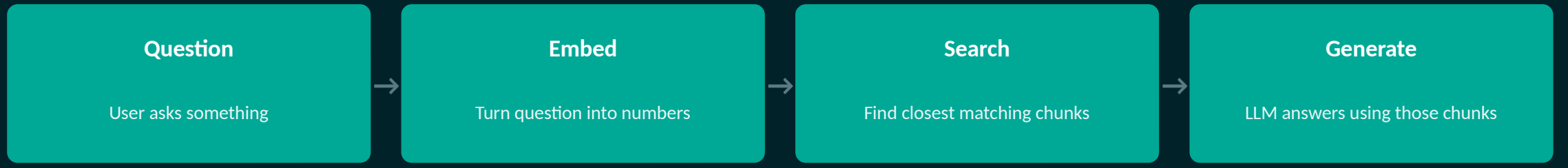
The Full RAG Flow

Two phases: preparing the knowledge once, then answering questions many times.

PHASE 1 — PREPARE (done once)



PHASE 2 — ASK (every time a user asks something)



That's the whole idea. No need to memorize the math — just remember: prepare once, then retrieve + generate every time someone asks.

PART 2 — HANDS ON

Practical — What We'll Build



Goal: A Company FAQ Bot

We'll give our agent a small piece of company knowledge (facts an LLM could never guess), and ask it real questions about that knowledge.

The Knowledge

A few sentences about a fictional company's policies

Embeddings

A free, local model — no extra API key needed

The Answer

Groq LLM answers using ONLY the retrieved facts

Continuing from before: same venv, same .env with GROQ_API_KEY. We just add two new packages today.

SETUP

Setup — Two New Packages

Open your existing project folder in VS Code, activate your venv, and run:

```
pip install langchain-huggingface sentence-transformers
```

langchain-huggingface	Lets LangChain use free, local embedding models from HuggingFace.
sentence-transformers	The actual library that turns text into embeddings, running on your own computer.

Why not use Groq for embeddings too?

Groq is built for fast chat responses, not embeddings. HuggingFace's free local model runs right on your computer — no extra API key, and completely free.

Note: the first run downloads a small model (~90 MB) — this only happens once.

HANDS-ON

Step 1 — Prepare the Knowledge

Create a file named rag_demo.py :

```
from dotenv import load_dotenv
load_dotenv()
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.documents import Document

# Our "knowledge" — facts no LLM could ever guess on its own
company_text = """
GreenTech was founded in 2015. We sell solar panels and home
battery systems. Our warranty is 10 years for panels and 5 years
for batteries. Support is available Monday to Friday, 9 AM to 6 PM.
"""

# Split it into small, searchable chunks
splitter = RecursiveCharacterTextSplitter(chunk_size=120, chunk_overlap=20)
chunks = splitter.split_text(company_text)
documents = [Document(page_content=chunk) for chunk in chunks]
```

EXPLANATION

Step 1 — What Just Happened?

`company_text`

In a real app, this would come from a PDF, website, or database — today we keep it simple with a plain string.

`chunk_size=120`

Each chunk holds about 120 characters — small enough to search precisely, big enough to keep context.

`chunk_overlap=20`

Chunks share a little text at the edges, so we don't accidentally cut an important fact in half.

`Document(page_content=...)`

LangChain's standard wrapper for a piece of text, so every tool downstream knows how to handle it.

HANDS-ON

Step 2 — Create Embeddings & Store Them

Add this below your Step 1 code:

```
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_core.vectorstores import InMemoryVectorStore

# Free, local model that turns text into embeddings
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

# A simple, in-memory database for those embeddings
vector_store = InMemoryVectorStore(embeddings)
vector_store.add_documents(documents)
```

This is Phase 1 (Prepare) from our theory slide — done! Our knowledge is now stored and searchable.

EXPLANATION

Step 2 — Line by Line

`HuggingFaceEmbeddings(...)`

Loads a small, free model that runs on your computer and turns any text into a list of numbers.

`all-MiniLM-L6-v2`

A popular, lightweight embedding model — fast, free, and good enough for most simple projects.

`InMemoryVectorStore(embeddings)`

Creates our searchable database, telling it which embedding model to use for every piece of text.

`add_documents(documents)`

Embeds every chunk from Step 1 and saves it inside the vector store, ready to be searched.

HANDS-ON

Step 3 — Ask a Question (Retrieve + Generate)

```
from langchain.chat_models import init_chat_model

question = "Batteries ki warranty kitne saal ki hai?"

# RETRIEVE: find the most relevant chunks
results = vector_store.similarity_search(question, k=2)
retrieved_text = "\n".join([doc.page_content for doc in results])

# GENERATE: let the LLM answer using ONLY that retrieved text
model = init_chat_model("groq:llama-3.3-70b-versatile")
prompt = f"Context:\n{retrieved_text}\n\nSawal: {question}"
answer = model.invoke(prompt)

print(answer.content)
```

HANDS-ON RESULT

Step 3 — This Is RAG, Completed

```
Run:   python rag_demo.py

Expected output:   "Batteries ki warranty 5 saal ki hai."
```

`similarity_search(question, k=2)` found the 2 chunks closest in meaning to the question, then we simply pasted that text into our prompt — the LLM never had to guess.

Try this: change the question to "Company kab bani thi?" or "Support kis time available hai?" and run again — the retrieved chunk changes each time, matching the new question.

EXERCISE

Your Turn — Grow the Knowledge Base

Add 2-3 more sentences to company_text about a new topic (returns, shipping, pricing — your choice).

```
company_text = """
GreenTech was founded in 2015. ... (your existing facts)
We offer free returns within 30 days of purchase.
Shipping is free for orders within the same city.
"""

question = "Kya returns free hain?"
```

Run the file again and watch the agent correctly pull in your NEW facts — without changing a single line of the RAG logic itself.

TROUBLESHOOTING

Common Errors and How to Fix Them

`ModuleNotFoundError`	Activate your venv, then run: pip install langchain-huggingface sentence-transformers
`First run feels slow / stuck`	Normal — it's downloading the small embedding model (~90 MB), only happens once.
`Answer says "I don't know"`	Check your question actually relates to something written in company_text.
`Wrong chunk retrieved`	Try increasing k in similarity_search(question, k=2) to pull in more chunks.
`Answer ignores the context`	Make the prompt more direct: "Answer using ONLY the context below."

WRAP UP

Recap of Today

1. RAG lets an LLM answer using information it was never trained on
2. An embedding turns text into numbers that capture its MEANING
3. A vector store saves embeddings and finds the ones closest to a new question
4. Prepare once (split, embed, store), then retrieve + generate every time someone asks
5. HuggingFaceEmbeddings gives us free, local embeddings — no extra API key needed
6. similarity_search() + a simple prompt is really all it takes to connect retrieval to the LLM



Your Agent Has Real Knowledge!

Next lecture: using LangGraph to formally connect everything we've built.